

Gemini CLI Enhancement Proposal

Ananya Kollipara (22kc34@queensu.ca)

Arlen Smith (22htl2@queensu.ca)

Christian Fiorino (22bqs2@queensu.ca)

Lillie Amos (22ryw1@queensu.ca)

Maia Turner (22pwg6@queensu.ca)

Vivian Webster (22nvp2@queensu.ca)

Enhancement Proposal

CISC 322 – Software Architecture

April 6th, 2026

Contents

0.0. Abstract	2
1.0. Introduction & Overview	2
2.0. Proposed Enhancement	3
2.1. Enhancement Implementation Alternative #1	3
2.2. Enhancement Implementation Alternative #2	4
3.0. SAAM Analysis	5
3.1. End-User Non-Functional Requirements	5
3.2. Developer Non-Functional Requirements	6
3.3. SAAM Analysis Conclusion	7
4.0. Use Cases	7
4.1. Use Case #1	7
4.2. Use Case #2	8
5.0. Enhancement Effects	10
5.1. Non-Functional Requirement Effects	10
5.2. Enhancement Effects on System Levels	11
6.0. Enhancement Effects	11
6.1. Impacted Directories & Flags	11
6.2. Testing Impacts	12
6.3. Potential Risks	12
7.0. Conclusion	13
8.0. Lessons Learned	13
9.0. Data Dictionary & Naming Conventions	14
9.1. Data Dictionary	14
9.2. Naming Conventions	14
10.0. AI Report	14
11.0. References	16

0.0. – Abstract

This report serves as a proposal for an enhancement to Gemini CLI. We put forth the idea of ‘Conversation Versioning’, which is a service that would allow the user to branch and choose how their conversation with Gemini’s chatbot would look by returning to previous conversation contexts and starting anew from there, while tracking old branches that can be returned to later.

In our writing we cover two hypothetical implementations for our enhancement. The final version will be chosen from the two based on their viabilities in end-user usability as well as developer maintainability. In two examples, we will demonstrate the conceptual architecture that goes into using our new feature, as well as possible ramifications the enhancement may have on existing components. Finally, we will be covering the testing process that would go into delivering our new product, ensuring on a concrete level that nothing implemented has unforeseen effects on the systems’ ability to run.

This is the third and final bit of writing from Group 8 in our analysis on Google’s Gemini CLI terminal agent. Coming from a conceptual analysis to scrutinizing the concrete to our own implementation, the final goal of this report is to demonstrate our group’s understanding of the systems and interconnection that goes into a significant piece of software such as this. Figures found while examining the implementation and use cases should show off our knowledge of how these components connect, and analysis of the ramification of this enhancement on NFRs should display an understanding of the concrete workings of this product. An accompanying video to this research will be posted on our website as well.

1.0. – Introduction and Overview

Our group, Teamm8s, has gathered information and experience across the past two reports on the conceptual and concrete architectures of Gemini CLI, and are now prepared to propose an enhancement to the architecture in the form of a brand-new feature. Throughout this report, our expertise on software architecture concepts will be on display as we explore our thought processes and analyze the effects of our proposed “Conversation Versioning” feature on the existing architecture of Gemini CLI, were it to be implemented.

First, we will explore our reasoning for deciding on this feature and how it meshes with the goals of Gemini CLI as a piece of software. We will then explore two different possibilities for implementing this feature and compare their effects on various non-functional requirements (NFRs) of stakeholders in a SAAM analysis. We will provide two use cases to explore how end-users interacting with this feature would propagate signals through the architectural components. We will explore the effects of implementing this enhancement on how NFRs are currently upheld and the high-level architecture of Gemini CLI. An exploration of impacted directories and flags, the impacts on testing, and potential risks in including this enhancement with our proposed method will be provided. Finally, our findings and conclusions drawn will be summarized at the end of our report.

A brief mention of the lessons our group learned in the creation of this report, explanations of our naming conventions used throughout, and a comprehensive report on our group’s use of artificial intelligence while writing this report by grading the contributions of the large language model we used. We hope our insights and expertise with our extended study of Gemini CLI’s architecture during this course has thoughtfully culminated in the contents of this report.

2.0 – Proposed Enhancement

The process we used to come up with a possible enhancement was to research what Gemini CLI's core design principles are and determine if a feature could be added to enhance one of their principles. One of Gemini CLI's design principles is its focus on providing a fulfilling user experience, allowing it to be more approachable by new users. However, we noticed that Gemini CLI is missing a critical component that makes working with an LLM easier. That being a way to record a chat session with the LLM, split a conversation into multiple different branches to allow its messages to be re-prompted if needed, and allow both the LLM and user to compare the two results of the two different prompts. This being said, Gemini CLI does have a form of recording chat sessions through its "Checkpoint" feature and session management. However, the Checkpoint feature is primarily used for agentic actions such as modifying files in the local system and does not save the session unless the user approves a tool that modifies the local system (*Checkpointing* 2026). This means that the Checkpoint feature is not utilized when Gemini CLI is used as a chatbot. Additionally, Gemini CLI's session management only records previous sessions and allows users to delete them. It doesn't allow users to alter or re-prompt the recorded sessions and compare the new result with the old one (*Session management* 2026). Which means that despite these pre-existing features, Gemini CLI is missing a method to record and re-prompt a chat session.

To resolve this issue, our proposed solution is to implement a "Conversation Versioning" feature into Gemini CLI. This feature utilizes the chats saved through the session management feature and allows the user to re-prompt specific LLM responses. Once the re-prompt is requested, Gemini CLI will create a copy of the session. With the copy deleting any conversations occurring after the re-prompted message and generating a new response. The user can then compare the results of the original and new prompts and delete either one if desired. Allowing the user to keep the old results and update the conversation to have the new prompt through two different branches.

The realization of this "Conversation Versioning" feature greatly improves Gemini CLI's user experience. As it allows users to experiment with different prompts in a more streamlined manner. The feature also helps with reducing and limiting hallucinations, as users can now re-prompt the LLM immediately if they realize its response is incorrect or non-sensical. The feature can additionally be integrated seamlessly into Gemini CLI through a new set of ~~commands~~ tools related to it. Further improving the user experience as the implementation would be consistent with Gemini CLI's use of commands.

We have devised two possible implementations for this enhancement. This first creates a new internal database that holds branching conversation that the context manager stored in `core\src\services\chatRecordingService.ts` could feed to and reference at the behest of the user's input. The second implementation creates a new tool that can be used to access Git for conversation versioning.

2.1. – Enhancement Implementation Alternative #1

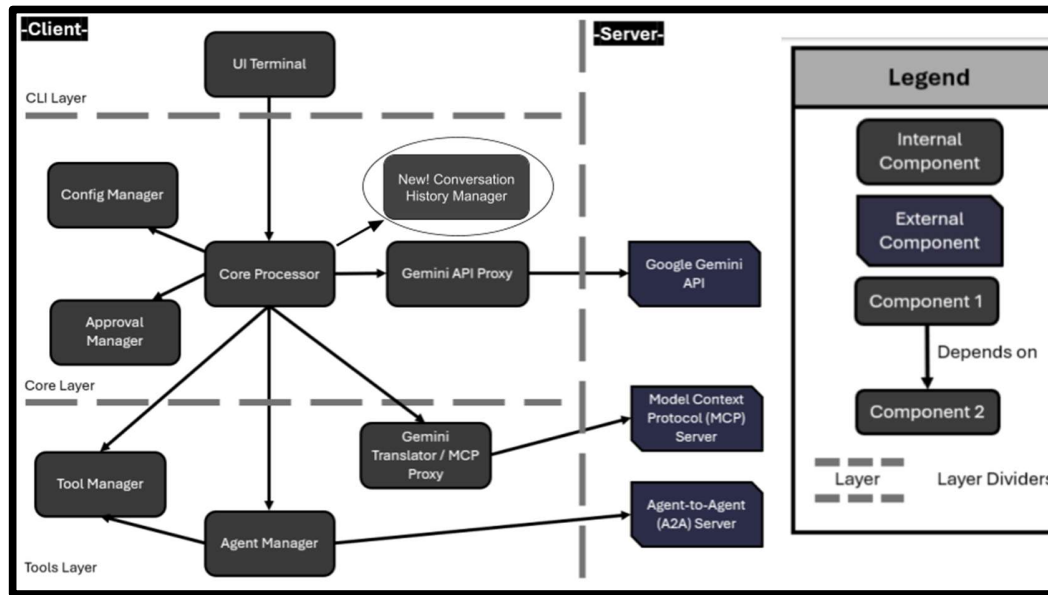


Figure 1. Updated Box-and-Line Diagram of Gemini CLI, with new history component

The new database component alternative is present in Figure 1; The new conversation history manager component will hold all previous prompts in a tree-like architecture. Whenever the user develops the conversation (inputs a new message), it is brought into the already existing component that manages the agent's current context. That information is then also passed to our new history manager component to continue that branch.

If the user wants to revisit an earlier point in the chat, they can ask to do so (likely by prompting the agent to show them the branching history stored in this new component, then highlighting a return point). The conversation within the context manager is then overwritten by the conversation history following and reaching the selected point in the branch's history. After this new point is selected, the next prompt from here will start a new branch in our history manager's tree structure.

This simpler alternative has the advantage over Git. Git provides plenty of functionality, much of which will never be useful for the purposes of this enhancement, such as merging. The user will never need to merge two conversation histories, since that would lead to a lot of confusion no matter how it is done. This implementation stays native to the system and never brings in functionality that we do not want. Advantages and disadvantages will be further discussed in the SAAM section up next.

2.2. – Enhancement Implementation Alternative #2

Another way to realise our implementation would be to have Gemini CLI utilize a Git-repository based approach to saving conversation history. This approach would save the user's Gemini CLI conversation history into a Git repository through commits and then use it to find the point in the user's session where they asked Gemini CLI a specific prompt. This implementation would be mostly defined within the existing Core Processor and Tool Manager components, as shown in Figure 2 below:

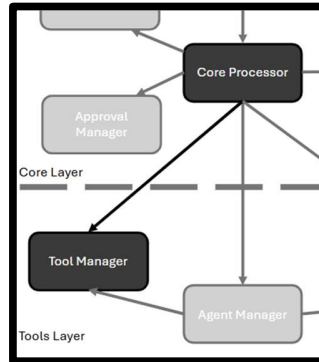


Figure 2. Updated Box-and-Line Diagram of Gemini CLI, with new Git tool

Like how Gemini CLI's current Checkpointing features stores snapshots of conversation history, a special git repository would be saved in the user's home directory. This git repository would contain the conversation history of the user's current session and would be updated after each prompt the user makes through Gemini CLI through git's add and commit commands. To ask Gemini CLI to re-prompt a specified prompt, the user would use a new tool for this purpose called the RepromptTool (defined in a file named reprompt.ts). This tool would be added into the existing Tool Manager component and would connect to the Git repository containing the session history to find the point where the user originally made the specified prompt using the git log command. Then, the current conversation before the user called the tool would be cloned to save the previous conversation history. The conversation at the point where the original prompt was called would then be restored using the git restore command, and the user would continue their session from that point. (*Git Reference*)

An advantage to using Git over the previous alternative is how this system would make use of already existing systems. Git already has well-known implementations of version control, which could make more sense to use over creating a new system in Gemini CLI for this purpose. Gemini CLI also already utilizes Git for several other purposes, such as for managing codebases, file contexts or workflows. This could make the implementation of this feature easier for the developers of Gemini CLI since they've already integrated Git into Gemini CLI in the past to create other features such as Git Worktrees. (*Git worktrees (experimental)*)

3.0. – SAAM Analysis Enhancement

In this analysis, we will compare our two implementation methods for our feature to determine which method should be utilized to implement the enhancement. We have identified our feature's major stakeholders to be the end-users and the Gemini CLI developers. As the end-users are those that will be utilizing Gemini CLI and its prompting capabilities to assist them with their tasks. Whereas the Gemini CLI developers require this feature to be easily maintainable and compatible with the rest of the system. Of these two stakeholder groups, we believe that the end-users are the more important stakeholder to consider as their number vastly outweighs the developers.

3.1. – End User Non-Functional Requirements

The main NFRs we have identified for the end-users are *usability* and *performance*. Starting with reviewing usability, it involves our enhancement being easy for the user to identify and select a

point previously in the conversation they'd like to go to, then return to that point. As for the performance NFR, it concerns how little space a given solution can take up, as well as its ability to switch between conversation contexts quickly. The below table compares the advantages and disadvantages of each implementation method with the end-user NFRs.

	Advantages	Disadvantages
Method #1 / Usability	• Component can be easily displayed as an image or ASCII characters. • Can utilize conversation IDs to identify each point in the conversation that a branch can be formed from, making it clearer to the user where a new conversation can be made.	• Giving each prompt an ID may be system intensive.
Method #2 / Usability	• Would make searching for and restoring previous prompts simple using git commands like 'git log' and 'git restore' on a conversation. (<i>Git-log Documentation; Git-restore Documentation</i>)	• Mapping a user's re-prompt request to a specific git commit may be difficult.
Method #1 / Performance	• The new component only takes up space equal to all written prompts. • A search and copy of a conversation tree would take $O(\log(n) + m)$ time, where (n) is the number of prompts needed to search the tree and (m) is the size of the conversation that needs to be copied to the context manager, which is relatively efficient. • The $O(m)$ efficiency is well overshadowed by the amount of time the agent takes to evaluate a prompt, as Gemini CLI re-evaluates the full context window for each response generated.	• There is a risk of holding too many conversations for an extended period of time if no limit is implemented.
Method #2 / Performance	• The git commands utilized in this enhancement implementation (those being 'git add' and 'git commit' to stash the conversation, 'git log' to find the commit the user wants, and 'git restore' to recover the previous conversation) would all be of $O(m)$. As they only affect the conversation file being modified. (<i>Git-add Documentation; Git-commit Documentation; Git-log Documentation; Git-restore Documentation</i>)	• The space complexity of this approach compared to Method #1 is much larger. Due to the added commits based on every prompt the user makes creating a new conversation file.

Table 1. Compares the implementation methods in the context of the end-user NFRs

3.2. – Developer Non-Functional Requirements

The primary NFR that needs to be considered for developers is *maintainability*. Which will be judged by the team's hypothetical ability to patch this functionality with bug fixes, new features, or general updates to how it runs. The following table discuss the advantages and disadvantages each implementation method has on maintainability.

	Advantages	Disadvantages
Method #1 / Maintainability	• Simplicity of component and data structure as well as minimal connections to other components results in very little to worry about.	• Display function may become complicated if UI changes
Method #2 / Maintainability	• Git itself is an external service that would be maintained outside of Gemini CLI, allowing it to maintain itself.	• New changes to Git could potentially break implementation functionality in Gemini CLI. • Bugs that appear within Git will affect the enhancement's functionality until it's fixed within Git • Git would include several unneeded features (such as branch merging or remote repository management) that could make this implementation more complex than needed. (<i>Git Reference</i>)

Table 2. Compares the implementation methods in the context of the developer NFR

3.3. – SAAM Analysis Conclusion

In the end, the internal logic database component alternative (method #1) is more space efficient, likely easier to display and access, and does not need extra tools to be properly maintained. We hold alternative 1 as our ideal implementation for the proposed enhancement of Conversation Versioning. It should serve to provide a simpler and more effective experience to users and developers alike.

4.1. – Use Case #1

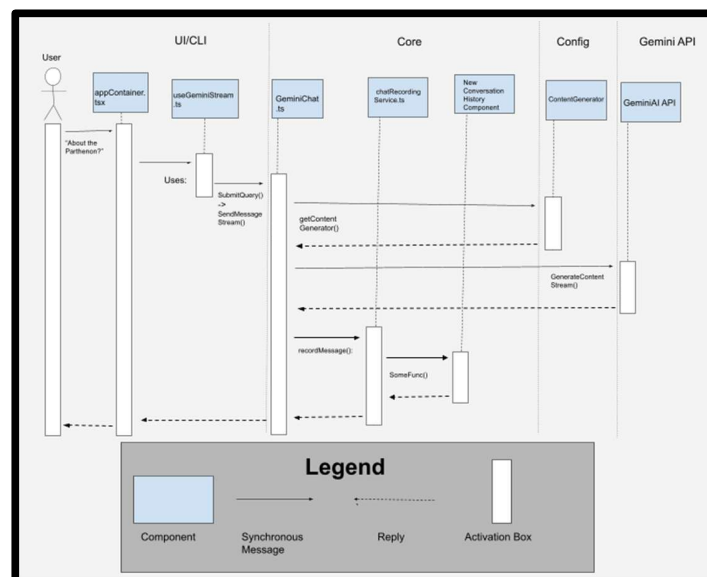


Figure 3. Sequence Diagram showing a prompt+response being added to the history component

The first use case we will look at here demonstrates how our new component will be fed information, along with how that information may look. Let's say the user has been curious to brush up on their history recently. They have been having a conversation with Gemini through the CLI. Previously, they have prompted the agent to indicate that the information they are searching for is specifically about architectural practices, then again to indicate that they only want information about said practices from the years 600 BC to 300 BC. Now the user wants to get specific. They ask the agent, "What can you tell me about the Parthenon in Greece?" This is how the information gets recorded into our new conversation history component:

The prompt is taken and assessed as before, up to the point where the response returns to GeminiChat.ts. Normally, as this information returns through the system to the user, it gets taken and recorded from GeminiChat.ts to chatRecordingServices.ts by its function recordMessage(), also in the core layer. This is the component that holds the current context of the conversation. It is usually used just to keep Gemini informed of what has previously been said, but this time the prompt and attached response will also be sent through some sending function to our new component: the conversation history manager. Both prompt and response will be condensed into a node and placed in a tree data structure in the new component, set as a child node to the node of the previous prompt in the conversation. While this tree looks a lot like the conversation in the current context component, its differences will become useful when we begin to branch later.

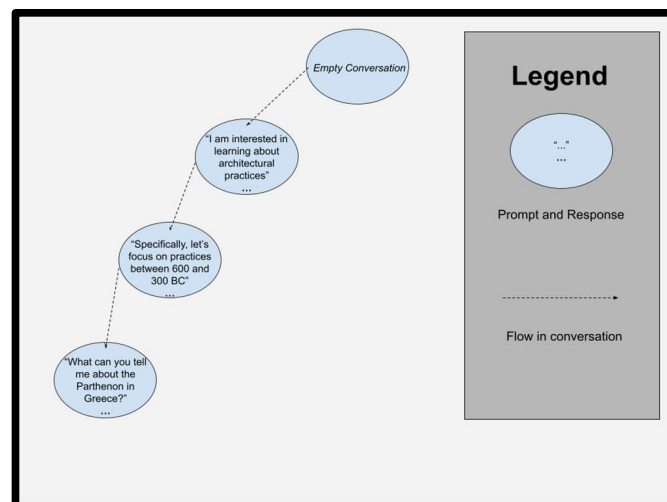


Figure 4. Visualization of the first conversation branch

4.2. – Use Case #2

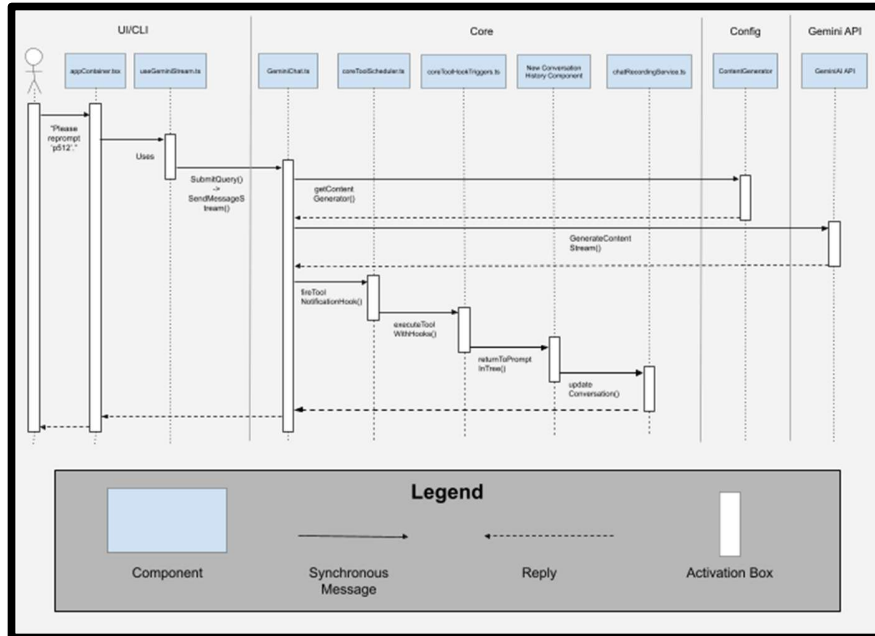


Figure 5. Sequence Diagram showing a new branch in the conversation being started

For our second use case, showcased in Figure 5, we will demonstrate how our enhancement would handle re-prompting and reverting conversations back to previous prompts. Continuing from the example used in the previous use case, let's say the user no longer wants to learn about practices only within 600 and 300 BC and instead wants to learn about practices from a different time period. But the user still would like to have the AI model use information it gave to them initially, so the user first would look through the IDs of previous prompts through a table to find the prompt ID of the prompt they want to re-prompt. The user finds the prompt to have an ID of 'p512', so the user asks the agent 'Please re-prompt 'p512''.

Through Gemini CLI's pre-existing functionality, the prompt would be processed through the Gemini API component as normal. The Gemini API component would then see that the user would like to re-prompt a previous prompt. It would call a tool for this purpose, leading to function calls being made in the coreToolScheduler.ts and coreToolHookTrigger.ts components. The tool called would check the newly implemented Conversation History component to access the prompt tree and find the specific ID that matches the ID of the prompt the user would like to re-prompt. Once the previous prompt has been found, a new branch would be made from this prompt, and the conversation history would be updated to include the history of the new conversation. The updated tree structure would be similar to the tree structure in Figure 6 below:

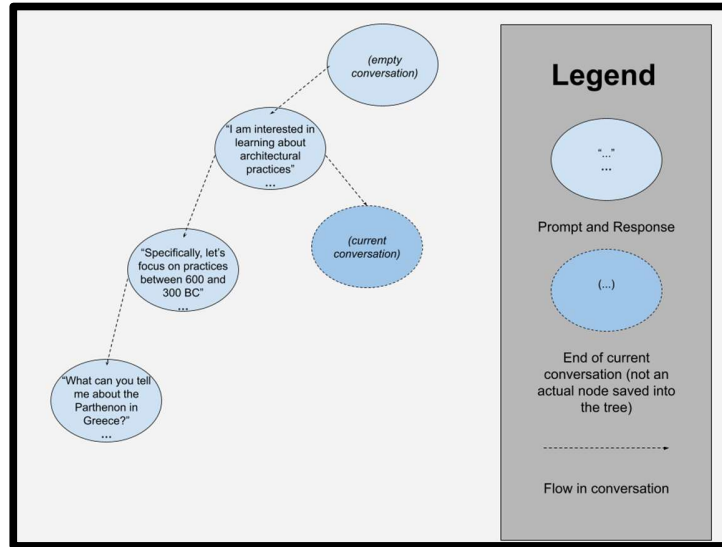


Figure 6. Visualization of the new conversation branch

5.0. – Enhancement Effects

This section of the report covers the effects the enhancement would have on the system. Including analyzing its effects based on specific non-functional requirements (NFRs) and the high-level and low-level architectures.

5.1. – Non-Functional Requirement Effects

By implementing this conversation versioning enhancement using our new database component, various positive and negative effects will be noticed on the NFRs for Gemini CLI as a whole. Some notable NFRs to explore this enhancement's ramifications on Gemini CLI are usability, performance, testability, maintainability, and evolvability.

Usability will be improved by this feature's usefulness, and at worst will slightly decline if users find the feature confusing to use or never bother to use it, in which case it would become unwanted feature bloat. Due to the existence of conversation versioning in most major LLM web applications, it is most likely the presence of this feature would become a usability improvement, especially for users who already have experience with standard LLM workflows.

Performance will take a slight hit due to the newly added ConversationHistoryManager component, as Gemini CLI will begin storing conversations for later use, which requires both space and time resources not used prior to this enhancement. With the efficient implementation outlined in section 3.0's SAAM analysis, the performance hit timewise should be negligible compared to the time it takes Gemini CLI to respond to a prompt. However, as was also mentioned in the SAAM analysis, certain design decisions regarding storage space optimization would need to be made to avoid storing large quantities of irrelevant conversations forever.

Testability will remain stable if our proposed multi-layer testing plan is followed, which will be more extensively documented later in section 6.2. The only case where testability will be reduced is there being either test cases we failed to consider in the creation of our testing plan, or if there are untestable cases in our implementation, which we do not believe there are.

Maintainability will be slightly reduced, as this feature is for the most part self-contained within the new ConversationHistoryManager component. As mentioned in section 3.0's SAAM analysis, any updates to the data structure holding the conversations should be easy to make, and as long as the contents of the current data structure can be ported to a new one should the need arise, no data would be lost in the update. Any UI changes would also be easy, as the bare minimum functionality is currently used in our implementation, allowing future developers to freely alter how the user interacts with this feature without changing the underlying architecture. This feature's use of the existing tool structure ensures developers can use, test, and modify it without learning a new system for doing so.

Evolvability remains stagnant for many of the same reasons explained in maintainability, with the only major issue being an adaptability issue all components with dependencies face. In our case, this issue stems from the new feature's interaction with the context window, as switching Gemini CLI's context to the selected branch in the conversation history is how this feature works. This means if how Gemini CLI's core layer handles the context window — or any dependency of the ConversationHistoryManager — were to change, then this newly added component would need to be updated to reflect those changes.

5.2. – *Enhancement Effects on System Levels*

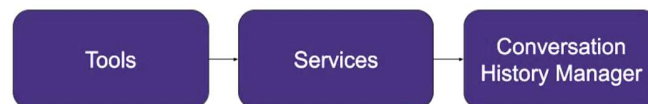


Figure 7. High Level Subsystems Affected By Enhancement

Figure 7 shows specifically the implementation changes demonstrated by this section. An overview of how this affects the entire architecture is shown in Figure 1. The core's services subsystem, specifically the ChatRecordingServices class in chatRecordingService.ts, needs to be updated to refer to our new Conversation History Manager component. Without prompting from the user, the effect of that ChatRecordingService will remain the same, adding to and displaying a list of all messages sent. That prompting can be given to Gemini CLI, which will call our new functions as a tool call like any other. This requires incorporation into the tools layer, both creating the script which can call our custom functions and also declare this script, so it is added to the Tool Registry. This is done for other scripts through the Config layer's config script, and so that is where we would add our as well. The functions that the user can call through this tool branch to a different path, which removes everything after it from the ChatRecordingService's return value, but keeps it stored in our tree structure, and it can be branched back to later. There is another function to view the current state of the tree our program has created. Due to the layered design philosophy, the changes we need to make do not affect that many files, as the software is modularly written.

6.0. – *Enhancement Impacts*

This section of the report discusses how the proposed enhancement affects the preexisting system. Including which directions it impacts, how it would be tested, and the potential risks of the enhancement.

6.1. – *Impacted Directories & Flags*

Our low-level architecture system names for the location are listed with the first word of each capitalized, with the actual filename in brackets beside each.

Impacted files:

- Core Processor/Services/chatRecordingService.ts (packages/core/src/services/chatRecordingService.ts)
- Config Manager/config.ts (packages/core/src/config/config.ts)

Impacted Directories:

- Core Processor/Services (packages/core/src/services)
- Tool Manager (packages/core/src/tools)
- Config Manager (packages/core/src/config)

6.2. – Testing Impacts

For testing the proposed enhancement implementation impacts of alternative 1 we will be focusing on information input and processing feature for prompting. The strategy will involve multiple layers of testing such as unit testing, end-to-end testing, integration testing, regression testing, and cross-platform testing. Unit testing will focus the introduced enhancement components expected core functionality seen in section 2.1 Figure 1 to verify the quality of each function, method and classes in isolation. For example, in the new Conversion History Manager component, test cases should cover creating a root node, adding child nodes, creating branches and multiple branches, updating and deleting nodes (and branches), traversing nodes, and edge cases such as depth limits and multiple-child limits. In addition to unit testing, end-to-end testing will mainly verify the end-to-end prompt correctness in the routing for each branched history node point. Integration testing will then simulate a Gemini CLI user prompting, the CLI agent's workflow, triggering prompt & response node creations with mocked branch node endpoints. To illustrate, after the input preprocessing it will check for prompt and response node creation, expected set up, and placement within the tree. Along with integration testing, regression testing will verify that there are no regressions from preceding versions of the Gemini CLI's tree like architecture and will still produces similar prompts and responses. For instance, in creating, setting, editing, and retrieving history of prompt & response nodes. For cross-platform testing, it will verify parsing, encoding formats and especially file handling across various operating systems. These multi-layered suites and systems of tests allow our feature to properly receive the user's input from the input processing systems for the Gemini CLI hybrid client & server layered design.

6.3. – Potential Risks

Despite the benefits of introducing this "Conversation Versioning" feature, it introduces a few risks that must be accounted for. One of these risks being performance-based. Specifically, the user's repeated use of this feature would eventually cause an overabundance of memory to be used due to the copied conversations, which could slow down the performance of the system. While the risk of this is small, as it would take thousands of conversation files for it to be a significant issue, it's important to have countermeasures to prevent the file sizes from becoming an issue. This could be achieved with a solution such as pruning the conversations within different branches based on how often they're used.

Along with this, our enhancement relies on the preexisting session management structure located in the Core Processor component. If this structure were to be updated by Gemini CLI's developers, then a maintainability-based risk would occur as our feature may become incompatible with the updated session management structure.

Lastly, due to the potentially sensitive information included in the conversations with Gemini CLI, a lack of encryption on the copied conversations could lead to a security-based risk. While Gemini CLI is primarily used as an agent for working on projects, it is still vital to have the copied conversations made from the "Conversation Versioning" feature be encrypted in some manner for users who utilize Gemini CLI more like a chatbot rather than an agent. Additionally, encrypting the copied files would assist in preventing any confidential information about a project from being leaked, which would benefit the users that use Gemini CLI as an agent.

7.0. – Conclusions

We believe that our proposed enhancement of Conversation Versioning would strongly benefit Gemini CLI. It boosts usability while having a low negative impact on performance and maintainability. The implementation found in implementation #1 keeps things simple and introduces only necessary functionality, allowing us to avoid too much intercommunication between scripts and dodge any relevant time-based performance issues. We chose implementation #1 over implementation #2 because using Git brought additional bloat into the enhancement. Our selected implementation takes up less space, uses fewer outside tools, and includes no redundant or unused functionality like Git's merge feature.

This enhancement should be very useful to those who want to use Gemini as a chatbot as much as a terminal agent by providing useful functionality present in the chatbot competition, while also keeping its individual identity as a terminal agent strong. Among the possible issues present, message security is a big one, though future encryption services could plug this hole. Overall, our enhancement benefits end-users without putting much strain on future developers.

8.0. – Lessons Learned

During this assignment, we learnt about the importance of NFRs and how they are utilized in system specifications. When initially learning about NFRs in lecture, our group thought they were superfluous additions to a project allowing developers to test their systems. But after utilizing NFRs in this deliverable, we understand they act more like a vision for the project that allows it to achieve a tangible goal. This makes NFRs one of the most important aspects when designing a system such as our proposed enhancement, as it ensures that the system has a clear purpose and can achieve some desired function without compromising on requirements.

Additionally, one thing we realized as we worked on our individual sections was that the work distribution was skewed. Two group members were working on most of the assignment's contents (specifically the implementations and SAAM analysis) that all the other sections relied upon. This led to the remaining four group members not having nearly as large of a workload as the other two. Based on this experience, we've learned the importance of dividing work evenly between group members and analyzing the workload more thoroughly beforehand before dividing tasks. Doing so not only creates a fairer environment for the whole group but also reduces the risk of only a subset of the group knowing most of the context for the assignment. If we were to do things differently for this deliverable, we would use our first meeting to ensure a

cleaner divide in each member's workload. We would also clarify with the professor and TAs roughly how long each section should be to get a better understanding of the effort required for each section, further allowing us to have an even work distribution between group members.

9.1. – Data Dictionary

- **Gemini AI | Gemini LLM** – Refers to one of Google's LLM models.
- **"core"** – package def
- **"cli"** – package def
- **"tools"** – package def
- **Tree** – A data structure consisting of nodes connected to each other through acyclic edges.
- **Git** – A system that provides version control for projects, organized by files and folders.

9.2. – Naming Conventions

- **CLI**: Command Line Interface
- **LLM**: Large Language Model
- **AI**: Artificial Intelligence
- **API**: Application Programming Interface
- **NFR(s)**: Non-Functional Requirement(s)
- **ASCII**: American Standard Code for Information Interchange

10.0. – AI Report

Before we selected what LLM model to utilize for our virtual member, we met as a group to determine where the LLM could be used within the assignment. As a result of the previous two assignments, we decided that we wanted the LLM to be more involved by completing tasks directly related to Gemini CLI's architecture. Along with this self-imposed requirement, we still wanted the LLM to boost our group's efficiency and accuracy where it can. Based on this, we assigned the LLM the following tasks:

- **Enhancement Idea Generation** – Given a prompt, the LLM will generate several possible enhancements that could be added to Gemini CLI. This is an ideal task for an LLM as they excel at ideation, being able to devise several ideas faster than we can think of. The LLM can also quickly search for what features in Gemini CLI already exist to ensure its ideas aren't already implemented, which could make it easier for us to ensure our enhancement is unique.
- **Report Evaluator** – Similar to the previous assignment, we would provide our report and the assignment rubric to the LLM, which will evaluate the report by ensuring all the required sections are present and providing a grade based on the rubric. As with the last assignment, this task is still ideal for an LLM due to their proficiency in logical comparisons and advanced reasoning. Allowing the LLM to provide effective feedback.

With these tasks defined, we needed an LLM that was good at idea generation, theoretically familiar with Gemini CLI to allow for unique enhancements, and had good logical reasoning skills. Based on these requirements, and our previous experience with it in A1, we decided to use Gemini 3 as our virtual member. Specifically, we used the Gemini 3 Thinking model as it utilizes a thinking process that enhances the LLM's logical reasoning and ideation abilities. We can also

view the LLM's thinking process, which allows us to more easily validate its outputs. Additionally, we used this model due to it possibly having more accurate information on Gemini CLI (as they both use the Gemini LLM models) and to compare how the Gemini 3 model does with the report evaluation task compared to the GPT-5.3 model we used in A2.

Similarly to the previous assignments, we decided to assign one member as the "prompt engineer". As only having one member work with the LLM simplifies the process of working with it and allows the other members to commit more time to their tasks. As with the previous assignments, the prompt engineer's job is to create prompts for the LLM, update the prompts to get the desired outputs if needed, and send the LLM's outputs to the rest of the group.

The prompt engineer wrote two prompts, one for the ideation and one for the report evaluation. Both prompts utilized a persona to focus the LLM's output, which meant that the prompts were inputted in separate versions of the LLM model for their respective tasks. Each prompt provided context to the LLM, where the ideation prompt gave background on what ideas it needed to generate and the report evaluation prompt included an uploaded set of files (those being a tailor-made report and assignment rubric) to perform its tasks. The outputs of each prompt were specified, with the ideation prompt asking the LLM to output a list of ideas that include an explanation of the idea and proof that it hasn't already been implemented in Gemini CLI. These prompts required few refinements to improve the quality (these refinements improving the clarity of the LLM's ideas and making their explanations sound less technical). The finalized version of the ideation prompt is displayed below:

You are a software engineer interested in analyzing complex projects and adding features to them. The most recent project you're analyzing as of March 2026 is the open-source tool Gemini CLI. Your task is to determine at least five new features that can be added to improve Gemini CLI's functionalities. These features shouldn't be small; they should change the architecture of the project. But the features shouldn't change the architecture too much such that the project must be completely restructured. Determining each feature will require analyzing Gemini CLI's current features (including their experimental ones) to determine what Gemini CLI is lacking in terms of functionality. The feature must not be incredibly technical such that a user with limited computer science knowledge can clearly understand it. For each feature you determine, you must provide a clear explanation of how the feature works and proof (through resources/links) that this feature has not been implemented yet as of March 2026.

We utilized two methodologies for validating the LLM's outputs. First, for the ideation prompt, we asked the LLM to provide links to evidence (such as GitHub commits and issues) that the ideas it generated were unimplemented in Gemini CLI. We also cross-referenced the LLM's output and its evidence with the Gemini CLI documentation to see if the possible enhancements the LLM generated were listed on the documentation. If the enhancement wasn't listed in the documentation and its evidence was consistent with the documentation, then we believed that the LLM's ideas were safe to consider. As for the second methodology used for the report evaluator, we cross-referenced the LLM's outputted feedback with the report and rubric to see if the feedback aligned within the context of the rubric and that what the LLM was grading was actually in the report. If the feedback did align with the rubric and what the LLM was discussing was in the report, then we considered the LLM's feedback to be valid.

Despite our attempt to integrate the LLM more into A3, we found that it didn't provide too much value to our final deliverable. While the LLM did generate sufficient enhancement ideas, we found that they were too technical and complex for us to properly consider, even after the prompt was revised to simplify the LLM's outputs. As such, we decided upon using an idea our group had come up with instead. Additionally, the LLM's evaluation of our report, while a fair evaluation, didn't provide any suggestions or criticisms that we weren't already aware of, reducing the potential importance of the LLM's evaluation. Based on this, we will grade the LLM's overall contribution as 5% as it didn't effectively contribute to the final deliverables.

As with the previous assignments, since only one member of our group worked with the LLM, it didn't create a lot of new work despite it not saving us a significant amount of time. We also found that the LLM didn't really influence our brainstorming and decision-making processes, as we were keener on the potential enhancements that we came up with rather than the LLM's ideas. With this reduced influence, the LLM didn't lead to any disagreements or challenges, but after this deliverable we did realize one thing about the LLM. That being that we didn't value its contributions. Partially because we wanted to learn the material ourselves, but also because we were worried that the LLM would output a hallucination that we wouldn't successfully identify despite our validation techniques. But when we're working with LLMs, the outputs it produces will never be fully trustworthy, and that's a risk we need to accept when using them. Going forward into our future endeavors, we will be more open to utilizing LLMs and their outputs, despite the inherent risk of hallucinations, and hopefully use them to improve our workflows and be more prepared for a future where we collaborate with LLMs rather than use them as tools.

11.0. – References

Checkpointing. Gemini CLI. (2026, March 19). <https://geminicli.com/docs/cli/checkpointing/>

Session management. Gemini CLI. (2026, March 20). <https://geminicli.com/docs/cli/session-management/>

Git worktrees (experimental). Gemini CLI. (2026, March 20). <https://geminicli.com/docs/cli/git-worktrees/>

Git Reference. Git. (n.d.). <https://git-scm.com/docs>

Git-add Documentation. Git. (n.d.). <https://git-scm.com/docs/git-add>

Git-commit Documentation. Git. (n.d.). <https://git-scm.com/docs/git-commit>

Git-log Documentation. Git. (n.d.). <https://git-scm.com/docs/git-log>

Git-restore Documentation. Git. (n.d.). <https://git-scm.com/docs/git-restore>

NFR Dictionary. Perforce. (2025, June 4). <https://www.perforce.com/blog/alm/what-are-non-functional-requirements-examples>